SCALABLE ADVANCED SOFTWARE SYSTEMS

Master's Level Assessment: NoSQL Architectures & Multi-Modal Databases

Architectures & Multi-Modal Databases

COM 769 | Knowledge Consolidation

01

FOUNDATIONS / ORIGINS OF NoSQL

How did the nomenclature and conceptual application of the term ‘NoSQL’ evolve between 1998 and 2009?

A) It was introduced in 1998 for document stores and repurposed in 2009 by Carlo Strozzi to describe graph databases.

B) It was coined by Carlo Strozzi in 1998 for an open-source relational database and repurposed in 2009 by Johan Oskarsson for distributed, non-relational databases.

C) It originated in the late 1960s to describe non-relational mainframes, but was formalised as ‘Not Only SQL’ by Strozzi in 2009.

D) It was created in 1998 to describe strict columnar databases and evolved in 2009 to describe multi-modal platforms.

E) Johan Oskarsson coined it in 1998 for early Key-Value stores, and Carlo Strozzi repurposed it in 2009 for NoREL languages.

01

ANSWER 01

A) It was introduced in 1998 for document stores and repurposed in

B) It was coined by Carlo Strozzi in 1998 for an open-source relational database and repurposed in 2009 by Johan Oskarsson for distributed, non-relational databases.

C) It originated in the late 1960s to describe non-relational mainframes,

D) It was created in 1998 to describe strict columnar

E) Johan Oskarsson coined it in 1998 for early Key-Value stores, and Carlo Strozzi repurposed it in 2009 for NoREL languages.

TECHNICAL RATIONALE

The term ‘NoSQL’ is a historical quirk. Carlo Strozzi originally coined it in 1998 for a lightweight, open-source relational database that simply did not expose a standard SQL interface. It was not until 2009 that Johan Oskarsson repurposed the term during a meetup to describe the emerging generation of open-source, distributed, non-relational data stores. Today, because many modern NoSQL systems support SQL-like query languages, the descriptor ‘NoREL’ (No Relations) would be architecturally more accurate.

02

From a systems development perspective, which of the following best characterises the structural implementation of semi-structured data?

A) It is stored in traditional database tables, strictly utilising foreign keys to infer hierarchy.

B) It consists entirely of opaque blobs retrieved via constant-time hashing algorithms.

C) It relies on tags and key/value pairs to define hierarchy, with its expression governed by a data serialisation language.

D) It is defined by the absence of any metadata, requiring complex search-match operations to parse.

E) It organises data into strict column families, requiring all rows to contain values for all columns.

02

ANSWER 02

A) It is stored in traditional database tables, strictly utilising foreign keys to infer hierarchy.

B) It consists entirely of opaque blobs retrieved via constant-time hashing algorithms.

C) It relies on tags and key/value pairs to define hierarchy, with its expression governed by a data serialisation language.

D) It is defined by the absence of any metadata, requiring

E) It organises data into strict column families, requiring all

TECHNICAL RATIONALE

Semi-structured data bridges the gap between rigid relational tables and entirely unstructured files. It uses internal tags and markers to make the organisation and hierarchy self-evident. Crucially for developers, the expression of this data is defined by serialisation languages or document formats (most commonly XML, JSON, or YAML). This structural format allows data stored in memory to be easily serialised, sent to distributed systems, parsed, and read by recipient microservices without requiring a static schema.

03

When architecting highly scalable systems across multiple nodes, which fundamental trade-off is typically leveraged by NoSQL databases to improve performance over RDBMS?

A) Increasing explicit relational modelling to pre-calculate node paths across horizontal servers.
B) Implementing stricter ACID axioms to ensure immediate global consistency across all clusters.
C) Shifting from an 'eventually consistent' strategy to a 'continuous availability' lock.
D) Reducing ACIDity, rules, and explicit relational modelling in favour of an 'eventually consistent' strategy.
E) Forcing a static schema to accelerate query read times across horizontal nodes.

03

ANSWER 03

A) Increasing explicit relational modelling to pre-calculate node paths across horizontal servers.
B) Implementing stricter ACID axioms to ensure immediate global consistency across all clusters.
C) Shifting from an 'eventually consistent' strategy to a 'continuous availability' lock.
D) Reducing ACIDity, rules, and explicit relational modelling in favour of an 'eventually consistent' strategy.
E) Forcing a static schema to accelerate query read times.

TECHNICAL RATIONALE

Relational databases struggle to scale horizontally because maintaining explicit relational modelling and strict ACID (Atomicity, Consistency, Isolation, Durability) properties across a distributed network introduces massive latency overheads. NoSQL databases achieve high scalability and read/write performance by deliberately relaxing these constraints. By removing explicit relational modelling and reducing rigid axioms, NoSQL systems often adopt an 'eventual consistency' model—vastly improving write throughput and read availability at massive big-data scale.

04

How do columnar databases (wide-column stores) functionally differentiate themselves from traditional RDBMS regarding row and column structure?

A) They strictly require all rows to contain values for every column defined in the table schema.
B) They store related rows in partitions and do not strictly require all rows to contain values for all columns.
C) They completely abandon the concept of columns, storing data exclusively as interconnected nodes.
D) They process data using recursive search-match operations rather than partition hashing.
E) They limit data types to opaque strings to ensure ultra-fast columnar retrieval.

04

ANSWER 04

A) They strictly require all rows to contain values for every column defined in the table schema.

B) They store related rows in partitions and do not strictly require all rows to contain values for all columns.

C) They completely abandon the concept of columns, storing data exclusively as interconnected nodes.

D) They process data using recursive search-match operations

E) They limit data types to opaque strings to ensure ultra-fast

TECHNICAL RATIONALE

While columnar databases (like Apache Cassandra) organise data into rows and columns, they introduce a vital twist: they are not bound by the strict schematics of an RDBMS. A row does not need to contain a value for every single column in the table, preventing the storage of millions of empty fields. Furthermore, they optimise for massive read/write speeds by organising related rows into partitions. These partitions are stored together on the same physical replicas, allowing the database to use hashing mechanisms to rapidly retrieve rows.

05

Which specific underlying mechanism allows Key-Value data stores to maintain high performance and constant time access, even when scaling to massive datasets?

- A) The deep inspection and indexing of the contents within the stored opaque blobs.
- B) The establishment of node-and-edge relationship records to map value locations.
- C) The use of document-oriented serialisation languages to flatten tables.
- D) The utilisation of hashing mechanisms to immediately retrieve associated values.
- E) The grouping of complex documents into continuously available static collections.

05

ANSWER 05

A) The deep inspection and indexing of the contents within the stored opaque blobs.
B) The establishment of node-and-edge relationship records to map value locations.
C) The use of document-oriented serialisation languages to flatten tables.
D) The utilisation of hashing mechanisms to immediately retrieve associated values.
E) The grouping of complex documents into continuously available static collections.

TECHNICAL RATIONALE

Key-value stores are among the least complex NoSQL architectures, making them incredibly fast. Their core performance advantage derives from hashing algorithms. When an indexed key is passed to the database, a hash function maps it directly to the associated value. This provides constant time access—meaning retrieval time remains flat even as the dataset scales massively. Because the database treats the value as an ‘opaque blob’ (a sequence of bytes it does not interpret or inspect), processing overhead is almost entirely eliminated.

06 DATABASE FAMILIES / DOCUMENT SCHEMAS

Unlike an RDBMS which requires a static, pre-defined schema, how does a Document Database handle data structure across a collection?

- A) It enforces a strict schema based on the precise layout of the first document entered into the collection.
- B) It relies on a flexible schema that is defined natively by the contents of each individual document.
- C) It requires a schema definition in a secondary relational table before JSON documents can be grouped.
- D) It avoids schemas entirely by storing documents as unstructured, unsearchable binary files.
- E) It uses a hashing algorithm to force all varied JSON documents into a unified tabular format upon retrieval.

06

ANSWER 06

A) It enforces a strict schema based on the precise layout of the first document entered into the collection.

B) It relies on a flexible schema that is defined natively by the contents of each individual document.

C) It requires a schema definition in a secondary relational table

D) It avoids schemas entirely by storing documents as unstructured, unseaochable binary files.

E) It uses a hashing algorithm to force all varied JSON documents

TECHNICAL RATIONALE

Document databases expand on the key-value concept but store values as complex, structured documents (typically JSON). The revolutionary aspect of this architecture is its flexible schema. The schema is not statically defined at the table level; rather, it is self-contained within the document itself. Within a single “collection”, one document might have 5 fields, while the next has an entirely different array of nested sub-documents. Because the database can natively inspect the document contents, it can perform advanced retrieval processing without requiring structural uniformity.

07 DATABASE FAMILIES / GRAPH ARCHITECTURE

What structural feature allows Graph Databases to drastically outperform relational databases when querying highly connected data?

- A) They store all interconnected data within a single, unpartitioned wide column.
- B) They package relationships into opaque blobs that load directly into constant-time caches.
- C) They completely avoid query languages, relying strictly on native serialisation parsing.
- D) They utilise nodes that contain explicit lists of relationship records (edges) pointing to other nodes.
- E) They enforce a uniform schema across all network collections to prevent recursion.

07

ANSWER 07

A) They store all interconmnected data within a single, unpartlioned wide column.
B) They package relationships into opaque blobs that load
C) They completely avoid query languages, relying strictly on native serialisation parsing.
D) They utilise nodes that contain explicit lists of relationship records (edges) pointing to other nodes.
E) They enforce a uniform schema across all network

TECHNICAL RATIONALE

In a relational database, finding connected data requires complex, time-consuming 'search-match' operations (JOINs), which degrade exponentially as data relationships deepen. Graph databases bypass this bottleneck entirely using a native graph architecture. Data items are stored as 'Nodes,' and their relationships are explicitly stee.' Because a node intrinsically contains a physical list of its relationship records pointing to other nodes, the database simply traverses these connections, offering blazing fast query performance for highly complex networks.

08 APPLIED ARCHITECTURE / USE CASES

An enterprise is attempting to anticipate software bugs by mapping and mining highly complex relationships across millions of historical customer support cases. Which database type and real-world precedent align with this requirement?

- A) Document Database (e.g., The Weather Channel using MongoDB).
- B) Columnar Database (e.g., Spotify using Cassandra).
- C) Key-Value Store (e.g., Twitter using Redis).
- D) Graph Database (e.g., Cisco using Neo4j).
- E) Multi-Modal Database (e.g., ASOS using Cosmos DB).

08

ANSWER 08

A) Document Database (e.g., The Weather Channel using MongoDB).
B) Columnar Database (e.g., Spotify using Cassandra).
C) Key-Value Store (e.g., Twitter using Redis).
D) Graph Database (e.g., Cisco using Neo4j).
E) Multi-Modal Database (e.g., ASOS using Cosmos DB).

TECHNICAL RATIONALE

Graph databases excel at establishing and interrogating complex data relationships across massive datasets, making them exceptional at finding commonalities or anomalies. Relational databases choke on these operations due to recursive JOINs. As highlighted in the core material, Cisco specifically utilises Neo4j, a leading Graph database, to mine their vast network of customer support cases. By modelling cases, symptoms, and software configurations as interconnected nodes, they can traverse the graph to rapidly anticipate and isolate software bugs.

09 PLATFORM SPECIFICS / AZURE COSMOS DB

Azure Cosmos DB provides single-digit millisecond response times at the 99th percentile. As a fully managed service, how does the platform handle capacity management to achieve this?

- A) It requires manual provisioning of hardware by database administrators during peak throughput loads.
- B) It enforces a unified relational schema to guarantee hardware efficiency at global scale.
- C) It uses cost-effective, serverless, and automatic scaling options that dynamically match capacity with demand.
- D) It relies entirely on client-side caching to artificially reduce latency and server load.
- E) It limits database provisioning to a single geographic region to prevent global distribution latency.

09

ANSWER 09

A) It requires manual provisioning of hardware by database administrators during peak throughput loads.
B) It enforces a unified relational schema to guarantee hardware efficiency at global scale.
C) It uses cost-effective, serverless, and automatic scaling options that dynamically match capacity with demand.
D) It relies entirely on client-side caching to artificially reduce latency and server load.
E) It limits database provisioning to a single geographic region to prevent global distribution latency.

TECHNICAL RATIONALE

Azure Cosmos DB is engineered natively for the cloud. To guarantee comprehensive SLAs—including single-digit millisecond response times—it removes the burden of manual database administration. It achieves this through elastic scale-out of storage and throughput, offering serverless and automatic scaling. This means the system autonomously detects application demand spikes and instantly scales compute resources to match that capacity, ensuring uninterrupted performance and cost-efficiency without requiring human intervention.

10

ARCHITECTURAL SYNTHESIS / MULTI-MODAL DEPLOYMENT

What is the primary architectural advantage of implementing a multi-model database, as demonstrated by Ulster University's sensor data platform?

- A) It isolates varying data models into separate physical servers to guarantee strict ACID compliance for sensor metrics.
- B) It automatically converts unstructured media files into highly structured SQL tables via proprietary APIs.
- C) It restricts developers to a single, hyper-optimised data model to simplify deployment across the university network.
- D) It supports multiple disparate data models against a single, integrated backend, simplifying development for billions of diverse data elements.
- E) It replaces all existing NoSQL databases with a strict, unified RDBMS schema designed for real-time telemetry.

١٥

ANSWER 10 / FINAL SYNTHESIS

A) It isolates varying data models into separate physical servers to guarantee strict ACID compliance for sensor metrics.
B) It automatically converts unstructured media files into highly structured SQL tables via proprietary APIs.
C) It restricts developers to a single, hyper-optimised data model to simplify deployment across the university network.
D) It supports multiple disparate data models against a single, integrate backend, simplifying development for billions of diverse data element
E) It replaces all existing NoSQL databases with a strict, unified RDBMS schema designed for real-time telemetry.

TECHNICAL RATIONALE

Historically, database systems forced a single data model. Multi-model databases disrupt this by allowing an enterprise to store and manipulate Key-Val Columnar, Document, and Graph data against a single, integrated backend. Ulste University leverages this to process billions of diverse sensor data elements.

FINAL TAKEAWAY: Advanced systems architecture is no longer about finding a single 'perfect' database. It is about matching the structural nuances of your data with the specific performance profile of the NoSQL paradigm, and unifying those distinct workloads through powerful, multi-modal cloud platforms. Assessment Comple